Low-C/N0 Tracking Improvement Plan

Date: 2026-07-19

Status: Proposed changes against `master` (`src/sdr_ch.c` v1.25). None of the changes in this document are implemented unless explicitly marked otherwise.

---

1. Objective

A 24 h comparison of GPS L1C/A tracking against a Septentrio mosaic-X5 (2026-07-11) showed Pocket SDR losing satellites below a reported ~32 dB-Hz, while the mosaic-X5 continued tracking to lower C/N0. The work below aims to:

- remove the low-C/N0 bias from the tracking C/N0 estimate;
- improve acquisition of signals for which a Doppler aid is available;
- extend coherent carrier tracking for secondary-code-synchronized pilots;
- preserve existing public APIs and the current E5 AltBOC tracking path;
- provide separately documented conservative and static low-C/N0 settings.

The target for the static low-C/N0 profile is approximately 24 dB-Hz for GPS L1C/A and 18 dB-Hz for supported pilot signals. These are validation targets, not guaranteed limits for every RF front end, oscillator or dynamic condition.

2. Current limitations

2.1 Biased tracking C/N0 estimator

`CN0()` in **src/sdr_ch.c** currently estimates

```
cn0_old = 10*log10(sumP / sumN / T)
```

where `sumP` includes both signal and prompt noise power. For an ideal correlator this gives

```
cn0_old = 10*log10(C/N0 + 1/T)
```

and therefore approaches a 30 dB-Hz floor for a 1 ms signal. For example, true 25 and 28 dB-Hz appear as approximately 31.2 and 32.1 dB-Hz.

2.2 Acquisition false-alarm floor

`sdr_corr_max()` subtracts the mean correlation power, but its maximum over all code/Doppler cells still has a noise-only extreme-value floor. In the measured L1C/A case (`fs = 4 MHz`, `t_acq = 0.02 s`, 41 Doppler bins corresponding to `max_dop = 10 kHz`) this floor was approximately 31 dB-Hz. The exact floor depends on the sample rate, code period, Doppler span, integration time and required false-alarm probability; 32 dB-Hz is not a universal lower bound.

2.3 Carrier-loop and loss-of-lock limits

For the current per-code-period Costas path, the expected PLI is approximately

```
PLI = (C/N0 * T) / (1 + C/N0 * T)
```

so `thres_pli = 0.25` reaches its structural limit near 25.3 dB-Hz for a 1 ms signal. A 10 Hz PLL bandwidth further raises the practical static tracking limit. Synthetic ramp tests must be repeated after implementation, but the existing result of approximately 27.8 dB-Hz for L1C/A is the baseline.

3. Planned implementation

3.1 Noise-subtracted tracking C/N0

Change the C and legacy Python estimators to:

```
S    = max(sumP - sumN, 0.01 * sumN)
cn0 = 10*log10(S / sumN / T)
```

This assumes that the N correlator is representative of the prompt noise power. Validation must include multiple signals, sample rates and real IF data to check noise coloration and residual code-correlation effects.

Threshold changes are policy changes as well as scale conversions; they must not be described as exact rescaling. The initial values to implement are:

item	current	planned	rationale
`THRES_CN0_U`	30.0 biased	25.0 true-scale	aligns with the current default PLI limit
`THRES_CN0_L6`	33.0 biased	32.5 true-scale	approximately equivalent for T = 4 ms
GUI reset `thres_cn0_u`	30.0 biased	25.0 true-scale	new estimator scale
`pocket_trk_default.conf` `thres_cn0_u`	30.5 biased	25.0 true-scale	new estimator scale
legacy Python lost threshold	32.0 biased	25.0 true-scale	intentional lower tracking gate
`THRES_CN0_L`	34.0 acquisition scale	unchanged initially	tune after acquisition Monte Carlo

Existing runtime configuration files require migration. An old `thres_cn0_u` value such as 30 will become an active true-scale cutoff after the estimator change.

3.2 Longer integration for Doppler-assisted acquisition

Add separate assisted-search options:

```
t_acq_ext
thres_cn0_ext
```

Use them only when an external Doppler aid is active (`acq->fd_ext != 0`), including recent-signal re-acquisition, other-signal assistance and predicted Doppler used by Fast Search. Unassisted acquisition continues to use `t_acq` and `thres_cn0_l`.

The initial candidate values are `t_acq_ext = 0.1 s` and `thres_cn0_ext = 30 dB-Hz` for conservative defaults. A value of 28 dB-Hz is reserved for the static low-C/N0 profile until false-alarm and re-acquisition probabilities are established by Monte Carlo testing.

The assisted acquisition threshold is:

```
max(thres_cn0_ext, signal_cn0_lost_threshold + 1 dB)
```

where the signal threshold is `thres_cn0_u`, or `THRES_CN0_L6` for L6D/E. This margin prevents deterministic cycling caused by the C/N0 gate only. PLI and loop-dynamic loss limits are separate; validation must demonstrate that a newly acquired channel remains locked under each supported settings profile.

The implementation must also:

- define valid positive ranges for `t_acq_ext` and `thres_cn0_ext`;
- use the existing search allocation and cleanup paths without leaks;
- preserve the current Fast Search Doppler-window behavior;
- test the near-zero-Doppler case because `fd_ext == 0` is the current no-assistance sentinel.

3.3 Extended coherent PLL updates for pilots

Add `sdr_sig_pilot()` for dataless pilot signals that have a known secondary code: L1CP, L5Q, L5SQ, G2OCP, G3OCP, E1C, E5AQ, E5ABQ, E5BQ, E6C, B1CP, B2AP and I1SP. Verify the list against the code and secondary-code generators during implementation.

After secondary-code synchronization and wipe-off:

- coherently sum prompt correlations over `K` code periods, with `K = max(1, round(t_coh/T))`;
- update the PLL using the coherent prompt and the actual interval `dt = K*T`;

- use `atan2(Q, I)` for the wiped pilot prompt;
- accumulate pilot PLI as `sumD/sumPs`, with both terms formed from the same coherent prompts;
- keep tracking C/N0 on the per-code-period prompt and N correlators;
- return immediately to the per-period Costas path if secondary-code sync is lost.

The stability check must use the effective interval, not the requested value:

```
b_pll * K * T < 0.4
```

Invalid `t_coh` values or unstable option combinations must be rejected or must disable coherent extension safely. The initial `t_coh` is 0.02 s.

`sec_pol` identifies the current half-cycle branch after secondary-code correlation; it is not itself the broadcast overlay sequence. The pilot path can maintain a full-cycle discriminator within a continuous lock interval, but the normal carrier integer ambiguity remains. Re-synchronization with reversed `sec_pol` after a fade can introduce a 0.5-cycle discontinuity and must set the appropriate loss-of-lock indication for carrier-phase users.

The E5ABQ implementation has changed substantially through `sdr_ch.c` v1.25. The pilot branch must be added on top of the current separate/pre-combined sideband logic and must not replace or regress its polarity calibration.

3.4 Runtime options and profiles

Add `t_acq_ext`, `t_coh` and `thres_cn0_ext` to `sdr_rcv_setopt()`, the `pocket_trk` configuration parser and the GUI System Options. Validate parsed values before using them in epoch-count or loop-stability calculations.

Keep two documented settings profiles:

option	conservative/dynamic	static low-C/N0
`t_acq`	0.02	0.02
`t_acq_ext`	0.10	0.10
`t_coh`	0.02	0.02
`thres_cn0_l`	34.0 initially	32.0 candidate
`thres_cn0_ext`	30.0 initially	28.0 candidate
`thres_cn0_u`	25.0	18.0
`thres_pli`	0.25	0.10
`b_pll`	10.0	5.0
`b_fll_w/b_fll_n`	5.0/2.0	5.0/2.0

The approximately 24 dB-Hz L1C/A and 18 dB-Hz pilot targets apply to the static low-C/N0 profile. The conservative profile intentionally retains more carrier dynamics and is not expected to reach the same limits. Initially, apply the static profile to `python/pocket_sdr.ini`; do not silently change all application defaults without separate dynamic testing.

The GUI layout change should be limited to what is required to expose the three new fields. A broader dialog refactoring is outside this tracking change.

4. Implementation sequence

1. Implement the C/N0 estimator and threshold migration in C and legacy Python; add deterministic estimator tests.
2. Implement and test the assisted-acquisition branch and new receiver options.
3. Add `sdr_sig_pilot()` and the coherent pilot PLL/PLI path on top of the current v1.25 tracking code.
4. Add configuration and GUI fields with input validation.
5. Run synthetic Monte Carlo, recorded-IF regression and dynamic tests.
6. Select final conservative and static-profile thresholds from the measured detection, false-alarm and tracking results.
7. Update `algo_desc.md`, user-facing option documentation and the final changed-file/version history only after implementation is complete.

5. Validation plan

5.1 C/N0 estimator

- Test true C/N0 from 15 to 45 dB-Hz in 1 dB steps with multiple random seeds.
- Cover at least T = 1, 4, 10 and 20 ms signals.
- Verify mean bias, standard deviation and the low-end estimator floor.
- Compare old and new estimates on recorded IF data against the reference receiver, allowing for front-end calibration differences.
- Confirm that RINEX SNR no longer has the mathematical 30 dB-Hz saturation. Do not claim receiver-to-receiver equality without calibration evidence.

5.2 Acquisition

- Run at least 1000 noise-only trials for each representative search-space size and settings profile.
- Measure detection probability at 2 dB steps around each threshold with at least 200 trials per point.
- Cover L1C/A and representative 4, 10 and 20 ms pilot signals.
- Vary `fs`, `max_dop`, aided-bin count and Doppler error.
- Verify recent-signal, other-signal and predicted-Doppler assistance.

- Check for acquire/lose cycling for at least 60 s near the threshold.
- Measure CPU time relative to the current unassisted search.

5.3 Tracking

- Repeat descending and ascending C/N0 ramps to expose hysteresis.
- Test L1C/A at both settings profiles and pilot signals with coherent extension enabled and disabled.
- Include static, oscillator-drift and representative kinematic dynamics.
- Force secondary-code loss and re-sync; verify fallback, recovery and carrier loss-of-lock indication.
- Confirm carrier phase continuity when `sec_pol` does not change and the expected 0.5-cycle handling when it does.
- Run specific E5ABQ regression tests for sideband selection, polarity calibration, C/N0 and carrier tracking.

5.4 Regression

- Build with the MSYS2 UCRT64 makefile environment.
- Run the existing C unit tests and relevant Python tests.
- Confirm unchanged acquisition/tracking behavior when the new pilot path is disabled or secondary-code sync is unavailable.
- Confirm that public function signatures remain compatible; adding `sdr_sig_pilot()` must not alter existing APIs.

6. Acceptance criteria

The work is complete only when:

- tracking C/N0 has no 30 dB-Hz mathematical floor for 1 ms signals;
- estimator bias is within 0.5 dB from 25 to 35 dB-Hz in the defined synthetic test setup, with variance also reported;
- selected acquisition thresholds meet a documented false-alarm probability and detection-rate target;
- no repeated acquire/lose cycle occurs in the threshold dwell tests;
- the static profile reaches approximately 24 dB-Hz for L1C/A and 18 dB-Hz for the supported pilots in the defined static synthetic test;
- conservative settings pass the defined dynamic test without regression;
- secondary-code loss and polarity reversal produce correct carrier-phase loss-of-lock behavior;
- E5ABQ and existing non-pilot tracking regressions pass;
- configuration migration and the meaning of both profiles are documented.

7. Files expected to change

- `src/sdr_ch.c`: C/N0 estimator, assisted acquisition, coherent pilot PLL/PLI and new option globals
- `src/sdr_code.c`: `sdr_sig_pilot()`
- `src/pocket_sdr.h`: pilot helper prototype and tracking state additions
- `src/sdr_rcv.c`: new runtime options and validation
- `python/sdr_ch.py`: noise-subtracted legacy C/N0 and intentional lost-gate update
- `python/sdr_opt.py`, `python/pocket_sdr.py`: new GUI fields and option wiring
- `python/pocket_sdr.ini`: static low-C/N0 profile
- `app/pocket_trk/pocket_trk_default.conf`: new options and true-scale C/N0 threshold migration; retain conservative loop settings initially
- relevant C/Python tests and algorithm/user documentation

8. Deferred work

- Bit-synchronized coherent extension for data signals such as L1C/A; the K-P bit synchronizer in `doc/update_sym_sync.md` can provide bit boundaries.
- FLL+DLL fallback that retains code observables after carrier-phase loss.
- Lowering the L6 CSK viability threshold using per-symbol CSK quality and erasure handling.